Übungsblatt 6

Optimierung

Skript: Abschnitte 4.4–4.6

Dieses Blatt übt den Stoff der sechsten Vorlesung. Teil A sind kurze Rechenaufgaben mit konkreten Zahlen: die Taylorformel in zwei Variablen, Hoch-, Tief- und Sattelpunkte, der Gradientenabstieg und Lagrange-Multiplikatoren. In Teil B kommen dieselben Themen in Lean: dieselben Funktionen, aber jetzt als Beweise, die der Compiler prüft.

Teil A: Auf Papier

Aufgabe A1 (Taylor in zwei Variablen)

Das Taylorpolynom k-ter Ordnung von f in p ist $T_k(h) = \sum_{j=0}^{k} \frac{1}{j!} (h \cdot \nabla)^j f(p)$, bis zur zweiten Ordnung also $T_2(h) = f(p) + \nabla f(p) \cdot h + \frac{1}{2}\, h^\top H_f(p)\, h$.

(a) Berechnen Sie T_2 für $f(x, y) = e^x \cos y$ in $p = (0, 0)$.

(b) Vergleichen Sie $T_2(0{,}1;\ 0{,}2)$ mit $f(0{,}1;\ 0{,}2)$, und berechnen Sie das Glied dritter Ordnung. Wie groß ist der Fehler von T_3?

(c) Sei $g(x, y) = x^2 y + 3y^2$ und $p = (1, 2)$. Berechnen Sie T_2 bei $h = (\frac{1}{10}, -\frac{1}{10})$ und vergleichen Sie mit $g(1{,}1;\ 1{,}9) = 13{,}129$. Warum ist T_3 exakt?

Hinweis: Zu (b): Es ist $f(0{,}1;\ 0{,}2) = 1{,}083141\ldots$, und in $(h \cdot \nabla)^3 f$ stehen die dritten Ableitungen: $f_{xxx}\, h_1^3 + 3 f_{xxy}\, h_1^2 h_2 + 3 f_{xyy}\, h_1 h_2^2 + f_{yyy}\, h_2^3$.

Aufgabe A2 (Hoch-, Tief- und Sattelpunkte)

Sei

$$f(x, y) = x^3 - 3x + y^3 - 3y.$$

(a) Berechnen Sie ∇f, alle kritischen Punkte und in jedem die Hesse-Matrix. Entscheiden Sie mit dem Hesse-Kriterium, wo ein Tiefpunkt, wo ein Hochpunkt und wo ein Sattelpunkt liegt.

(b) Bestätigen Sie den Tiefpunkt ohne Hesse-Kriterium: Zeigen Sie $f(1 + h, 1 + k) - f(1, 1) = h^2\,(3 + h) + k^2\,(3 + k)$. Auf welcher Menge ist damit $f \geq f(1, 1)$? Wie folgt daraus der Hochpunkt?

(c) Bestätigen Sie den Sattelpunkt $(1, -1)$ mit den Schnittkurven $t \mapsto f(1 + t, -1)$ und $t \mapsto f(1, -1 + t)$. Warum ist auch $(-1, 1)$ ein Sattelpunkt?

(d) Hat f ein globales Minimum oder Maximum?

(e) Für $q(x, y) = x^2 + 3xy + y^2$ haben beide Schnittkurven längs der Achsen, $q(t, 0) = t^2$ und $q(0, t) = t^2$, in $t = 0$ ein Minimum. Was sagt das Kriterium in zwei Dimensionen über $(0, 0)$? Berechnen Sie $q(t, -t)$ und die Eigenwerte der Hesse-Matrix von q.

Hinweis: Zu (b): $f(-x,-y) = -f(x,y)$. Zu (c): $f(x,y) = f(y,x)$. Zu (e): Die Eigenvektoren sind $(1,1)$ und $(1,-1)$.

Aufgabe A3 (Gradientenabstieg)

Der Gradientenabstieg mit Schrittweite $t > 0$ ist $x_{k+1} = x_k - t\,\nabla f(x_k)$.

(a) Sei $f(x) = (x-2)^2$, $t = \frac{1}{4}$ und $x_0 = 6$. Berechnen Sie x_1, x_2, x_3 und zeigen Sie $x_{k+1} - 2 = \frac{1}{2}(x_k - 2)$. Wie viele Schritte braucht man, bis $|x_n - 2| \le \frac{1}{1000}$ ist?

(b) Für welche Schrittweiten t konvergiert das Verfahren in (a) für jeden Startwert? Was passiert bei $t = 1$ und bei $t = \frac{6}{5}$ mit $x_0 = 6$?

(c) Sei nun $f(x,y) = \frac{1}{2}(x^2 + 4y^2)$ mit Minimum in $(0,0)$. Berechnen Sie ∇f, dann zwei Schritte mit $t = \frac{1}{5}$ von $(x_0, y_0) = (4,1)$ aus, und geben Sie (x_n, y_n) für alle n an.

(d) Für welche t konvergiert das Verfahren in (c) von jedem Startwert aus gegen $(0,0)$? Was passiert bei $t = \frac{1}{2}$?

(e) Sei $\rho(t) = \max\{\,|1-t|,\ |1-4t|\,\}$; dann ist $\|(x_n,y_n)\| \le \rho(t)^n\,\|(x_0,y_0)\|$. Für welches t ist $\rho(t)$ am kleinsten? Wie viele Schritte garantiert die Schranke bis $\|(x_n,y_n)\| \le \frac{1}{1000}\,\|(x_0,y_0)\|$ mit diesem t, wie viele mit $t = \frac{1}{5}$?

Hinweis: In jedem Schritt wird der Fehler (bzw. jede Koordinate) mit einem festen Faktor multipliziert. Zu (e): $|1-t|$ und $|1-4t|$ über t skizzieren.

Aufgabe A4 (Lagrange)

(a) Minimieren Sie $\varphi(x,y) = x^2 + y^2$ unter der Nebenbedingung $f(x,y) = x + 2y = 5$: Prüfen Sie $\nabla f \neq 0$, stellen Sie $\nabla\varphi = \lambda\,\nabla f$ auf und bestimmen Sie λ und den Kandidaten p. Welchen Wert hat φ dort?

(b) Kontrollieren Sie das ohne Lagrange: Setzen Sie $x = 5 - 2y$ in φ ein.

(c) Bestimmen Sie die Lagrange-Kandidaten für $\varphi(x,y) = x+y$ unter $x^2+y^2 = 2$ und entscheiden Sie, wo das Maximum und wo das Minimum liegt.

Teil B: In Lean

Öffnen Sie die Datei `lean/Blatt06.lean` im Editor (VS Code mit der Lean-4-Erweiterung, in diesem Repository mit Mathlib), und ersetzen Sie jedes `sorry` durch einen Beweis; der Infoview zeigt Ziel und Kontext. Die Datei ist fertig, wenn Lean keine Warnung „declaration uses ‚sorry'" mehr meldet. Die Beweise, auf die die Kommentare der Datei mit Namen verweisen, stehen in `code/Vorlesung06.lean` und, für `betrag_nicht_klein_o`, auf Blatt 5. Beim Suchen helfen `exact?` und `apply?`.

Aufgabe B1 (Hoch-, Tief- und Sattelpunkte: zu A2 (b) und (c))

Punkte der Ebene sind in diesem Teil Paare `p` mit den Koordinaten `p.1` und `p.2`; `landschaft` ist die Funktion f aus A2. `IsLocalMin f p` heißt: Schließlich längs $\mathcal{N}(p)$ ist $f(p) \le f(q)$; `IsLocalMax f p` ebenso mit $f(p) \ge f(q)$. Auf Paaren rechnet Mathlib mit der Maximumsnorm: Die Kugel vom Radius δ um p ist das Quadrat aus allen q mit $|q_1 - p_1| < \delta$ und $|q_2 - p_2| < \delta$.

(a) Für $x, y \ge -2$ ist $f(1,1) \le f(x,y)$.

(b) f hat in $(1,1)$ ein lokales Minimum.

(c) f hat in $(1,-1)$ weder ein lokales Minimum noch ein lokales Maximum.

Hinweis: (a) ist die Rechnung aus A2 (b) mit $x = 1+h$ und $y = 1+k$: $f(x,y) - f(1,1) = (x-1)^2(x+2) + (y-1)^2(y+2)$. Diese Gleichung zeigt `ring`, die Vorzeichen der Summanden `mul_nonneg` und `sq_nonneg`. (b) `filter_upwards` mit der Kugel vom Radius 3 um $(1,1)$; `Prod.dist_eq`, `max_lt_iff`, `Real.dist_eq` und `abs_lt` machen aus „p liegt in der Kugel" Ungleichungen für die Koordinaten. (c) geht wie B3 (b) auf Blatt 5: `Metric.eventually_nhds_iff` liefert eine Kugel vom Radius δ; die Punkte $(1,-1+t)$ und $(1+t,-1)$ mit $t = \min(\delta/2, 1)$ liegen darin. Ungleichungen mit Produkten erledigt `nlinarith`, wenn man ihm $t^2(3-t) > 0$ bzw. $t^2(3+t) > 0$ als `mul_pos` … mitgibt.

Lean `lean/Blatt06.lean`

```
/-- Die Funktion aus A2, `f(x, y) = x³ - 3x + y³ - 3y`. Punkte der Ebene sind
    Paare `p` mit den Koordinaten `p.1` und `p.2`. -/
def landschaft (p : ℝ × ℝ) : ℝ := p.1 ^ 3 - 3 * p.1 + (p.2 ^ 3 - 3 * p.2)

-- `IsLocalMin f p` heisst: schliesslich laengs `𝒩 p` ist `f p ≤ f q`;
-- `IsLocalMax f p` ebenso mit `f q ≤ f p`. Auf Paaren rechnet Mathlib mit
-- der Maximumsnorm: Die Kugel vom Radius δ um p ist ein Quadrat.

/-- Aufgabe B1 (a) (A2 (b)): Fuer `x, y ≥ -2` ist `f(1, 1) ≤ f(x, y)`. -/
theorem landschaft_abschaetzung (p : ℝ × ℝ) (hx : -2 ≤ p.1)
    (hy : -2 ≤ p.2) : landschaft (1, 1) ≤ landschaft p := by
  -- Nuetzlich: die Rechnung von Papier als Gleichung `have e : ... := by
  -- simp only [landschaft]; ring`, dann mul_nonneg, sq_nonneg, linarith
  sorry

/-- Aufgabe B1 (b) (A2 (b)): `f` hat in `(1, 1)` ein lokales Minimum. -/
theorem landschaft_tal : IsLocalMin landschaft (1, 1) := by
  -- Nuetzlich: filter_upwards mit Metric.ball_mem_nhds (die Kugel vom
  -- Radius 3 um (1, 1)), Metric.mem_ball, Prod.dist_eq, max_lt_iff,
  -- Real.dist_eq, abs_lt, linarith; dann (a)
  sorry

/-- Aufgabe B1 (c) (A2 (c)): In `(1, -1)` hat `f` weder ein lokales Minimum
    noch ein lokales Maximum: ein Sattelpunkt. -/
theorem landschaft_sattel :
    ¬ IsLocalMin landschaft (1, -1) ∧ ¬ IsLocalMax landschaft (1, -1) := by
  -- Nuetzlich: wie betrag_nicht_klein_o auf Blatt 5 (B3 (b)):
    Metric.eventually_nhds_iff
```

```
  -- liefert ein δ > 0. Die Punkte (1, -1 + t) und (1 + t, -1) mit
  -- t = min (δ / 2) 1 liegen in der Kugel vom Radius δ (lt_min, half_pos,
  -- min_le_left, min_le_right, half_lt_self, Prod.dist_eq, max_lt_iff,
  -- Real.dist_eq, abs_of_pos). Dann simp only [landschaft] und nlinarith mit
  -- dem Hinweis mul_pos (mul_pos ht ht) _ (mit ht : 0 < t) fuer
  -- t²(3 - t) > 0 bzw. t²(3 + t) > 0
  sorry
```

Aufgabe B2 (Gradientenabstieg: zu A3 (c) und (d))

Punkte der Ebene sind in diesem Teil Paare `p` mit den Koordinaten `p.1` und `p.2`. Auf Paaren rechnet Mathlib mit der Maximumsnorm statt der euklidischen Norm; für die Konvergenz macht das keinen Unterschied – beide Male heißt sie: Konvergenz beider Koordinaten. Vorgegeben sind der Gradient von $f(x,y) = \frac{1}{2}(ax^2 + by^2)$ als `gradF` (A3 hat $a = 1$, $b = 4$; dass er der Gradient von f ist, beweisen wir in Lean nicht), ein Schritt `gdSchritt2` und die Iteration `gdIter2`, gebaut wie `gdSchritt` und `gdIter` in der Vorlesung. Vorgegeben ist auch das Lemma `gd2_schritt`: Ein Schritt multipliziert die Koordinaten mit $1 - ta$ bzw. $1 - tb$.

(a) Die geschlossene Form aus A3 (c): $x_n = (1 - ta)^n\, x_0$ und $y_n = (1 - tb)^n\, y_0$.

(b) $|1 - ta| < 1$ und $|1 - tb| < 1$ gelten genau dann, wenn $0 < t < 2/b$ – für $0 < a \le b$.

(c) Für $0 < t < 2/b$ konvergiert das Verfahren von jedem Startwert aus gegen $(0,0)$ – die Richtung „⇐“ von A3 (d).

Hinweis: (a) geht wie `gd_q_fehler` aus der Vorlesung; nach `ext` ist für jede Koordinate eine Gleichung zwischen reellen Zahlen übrig. (b) geht wie `schrittweite` aus der Vorlesung. In (c) liefert (b) die beiden Beträge < 1; dann weiter wie in `gd_q_tendsto`. `Tendsto.prodMk_nhds` setzt die Grenzwerte der beiden Koordinaten zu einem Grenzwert in der Ebene zusammen.

Lean `lean/Blatt06.lean`

```
/-- Der Gradient von `f(x, y) = (a x² + b y²)/2` -- fuer `a = 1`, `b = 4` die
    Funktion aus A3. Punkte der Ebene sind Paare `p` mit den Koordinaten `p.1`
    und `p.2`. -/
def gradF (a b : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := (a * p.1, b * p.2)

/-- Ein Schritt des Gradientenabstiegs, `p - t ∇f(p)` (wie gdSchritt). -/
def gdSchritt2 (a b t : ℝ) (p : ℝ × ℝ) : ℝ × ℝ := p - t • gradF a b p

/-- Die Iteration `p₀, p₁, p₂, ...` (wie gdIter in der Vorlesung). -/
def gdIter2 (a b t : ℝ) (p₀ : ℝ × ℝ) : ℕ → ℝ × ℝ
  | 0 => p₀
  | n + 1 => gdSchritt2 a b t (gdIter2 a b t p₀ n)

/-- Vorgegeben: Ein Schritt multipliziert die Koordinaten mit `1 - t a` bzw.
    `1 - t b`. `ext` zerlegt eine Gleichung zwischen Paaren in zwei
    Gleichungen, eine fuer jede Koordinate. -/
theorem gd2_schritt (a b t : ℝ) (p : ℝ × ℝ) :
    gdSchritt2 a b t p = ((1 - t * a) * p.1, (1 - t * b) * p.2) := by
  ext
  · simp [gdSchritt2, gradF]
    ring
  · simp [gdSchritt2, gradF]
    ring

/-- Aufgabe B2 (a) (A3 (c)): die geschlossene Form. -/
theorem gd2_formel (a b t : ℝ) (p₀ : ℝ × ℝ) (n : ℕ) :
```

```
    gdIter2 a b t p₀ n = ((1 - t * a) ^ n * p₀.1, (1 - t * b) ^ n * p₀.2) := by
  -- Nuetzlich: wie gd_q_fehler in der Vorlesung, mit gd2_schritt; fuer
  -- Gleichungen zwischen Paaren hilft ext (siehe gd2_schritt)
  sorry

/-- Aufgabe B2 (b) (A3 (d), Rechnung): Beide Faktoren haben genau dann Betrag
    '< 1', wenn '0 < t < 2/b'. -/
theorem schrittweite2 (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) :
    (|1 - t * a| < 1 ∧ |1 - t * b| < 1) ↔ (0 < t ∧ t < 2 / b) := by
  -- Nuetzlich: wie schrittweite in der Vorlesung: abs_lt, lt_div_iff₀
  -- (dafuer erst 0 < b zeigen), constructor, rintro, nlinarith
  sorry

/-- Aufgabe B2 (c) (A3 (d), eine Richtung): Fuer '0 < t < 2/b' konvergiert das
    Verfahren von jedem Startwert aus gegen das Minimum '(0, 0)'. -/
theorem gd2_konvergenz (a b t : ℝ) (ha : 0 < a) (hab : a ≤ b) (ht : 0 < t)
    (ht' : t < 2 / b) (p₀ : ℝ × ℝ) :
    Tendsto (gdIter2 a b t p₀) atTop (𝓝 (0, 0)) := by
  -- Nuetzlich: schrittweite2; aus gd_q_tendsto in der Vorlesung
  -- tendsto_pow_atTop_nhds_zero_of_norm_lt_one, Real.norm_eq_abs und
  -- Tendsto.mul_const; Tendsto.prodMk_nhds; funext (gd2_formel a b t p₀)
  -- ist die Formel als Gleichung zwischen Folgen
  sorry
```

Aufgabe B3 (Lagrange: zu A4 (a) und (b))

Lagrange liefert den Kandidaten; dass er wirklich das Optimum ist, bestätigt eine elementare Ungleichung, wie bei $xy \le \frac{1}{4}$ unter $x + y = 1$ in der Vorlesung.

(a) Auf der Geraden $x + 2y = 5$ ist $x^2 + y^2 \ge 5$.

(b) Der Wert 5 wird nur im Lagrange-Punkt $(1, 2)$ angenommen.

Hinweis: Es ist $5\,(x^2 + y^2) = (x + 2y)^2 + (2x - y)^2$. Geben Sie `nlinarith` den Hinweis `sq_nonneg (2 * x - y)`. In (b) ist dann $(2x - y)^2 = 0$, und aus `pow_eq_zero_iff` folgt $2x - y = 0$.

Lean lean/Blatt06.lean

```
/-- Aufgabe B3 (a) (A4 (a)): Auf der Geraden 'x + 2y = 5' ist 'x² + y² ≥ 5';
    der Lagrange-Kandidat '(1, 2)' liefert den Wert '5'. -/
theorem abstand_gerade (x y : ℝ) (h : x + 2 * y = 5) : 5 ≤ x ^ 2 + y ^ 2 := by
  -- Nuetzlich: wie xy_le_quarter in der Vorlesung: nlinarith mit dem Hinweis
  -- sq_nonneg (2 * x - y); denn 5·(x² + y²) = (x + 2y)² + (2x - y)²
  sorry

/-- Aufgabe B3 (b) (A4 (a)): Der Wert '5' wird nur in '(1, 2)' angenommen. -/
theorem abstand_gerade_eindeutig (x y : ℝ) (h : x + 2 * y = 5)
    (hmin : x ^ 2 + y ^ 2 = 5) : x = 1 ∧ y = 2 := by
  -- Nuetzlich: Aus der Gleichung 5·(x² + y²) = (x + 2y)² + (2x - y)² folgt
  -- (2x - y)² = 0 (nlinarith), also 2x - y = 0 (pow_eq_zero_iff); dann linarith
  sorry
```